

What You Will Learn

This guide walks through every line of SmartDocs AI — an AI-powered PDF Q&A; tool built with Python, Streamlit, and Claude API. You will understand not just WHAT each line does, but WHY it exists. Each version builds on the last, showing how a real product evolves from prototype to production.

V1 — Prototype	Core working app. 50 lines. Upload PDF, ask Claude, get answer.
V2 — Production	Error handling, security, client API key input, UI polish.
V3 — Product	Page range selection, pdfplumber, branding, testing suite.
Testing	Automated QA suite achieving 100/100 score.
Concepts	Python imports, session state, API calls, error handling, deployment.

The Tech Stack

SmartDocs uses four technologies. Understanding each one before reading the code makes everything clearer.

Python	The programming language. Everything is written in Python 3.11.
Streamlit	Turns Python scripts into web apps instantly. No HTML needed. Every time a user clicks anything — Streamlit reruns your entire Python file from top to bottom.
Claude API	Anthropic's AI model accessed programmatically. You send text, Claude sends back an intelligent answer. Stateless — no memory between calls.
pdfplumber	Reads PDF files and extracts text page by page. Better than PyPDF2 for complex PDFs.

Key Concepts You Must Understand

1. Stateless API

Claude remembers NOTHING between API calls. Every call is completely fresh. This is why we manually send the entire conversation history every time — to simulate memory.

■ Java equivalent: Like a REST endpoint. The server remembers nothing. You must send everything needed in each request.

2. Streamlit Rerun Model

Every user action (click, upload, type) causes Streamlit to rerun your ENTIRE Python file from line 1. This means normal variables reset every rerun. Only `st.session_state` survives across reruns.

■ Think of it like: every button click creates a brand new Python process that runs your whole file again.

3. Session State

`st.session_state` is a dictionary that persists across reruns. Anything you store here survives the next rerun. This is how SmartDocs remembers your uploaded PDF and chat history between questions.

4. Environment Variables

API keys must never be hardcoded in code. They are stored in a `.env` file locally, or in Streamlit's Secrets manager when deployed. The code reads them at runtime without ever seeing them in the source code.

■ Java equivalent: `application.properties` in Spring Boot. Secrets stay outside your code.

V1 — Core Working App

V1 is the minimum working version. 50 lines. One goal: upload a PDF, ask a question, get an answer from Claude. No error handling. No security. No polish. Just proof that the idea works.

Goal	Upload PDF → ask question → get answer
Lines of code	~50
Libraries	streamlit, anthropic, PyPDF2, python-dotenv
Status	Working prototype, not for real users

Dependencies — What To Install

Run these commands in your terminal before writing any code:

```
pip install streamlit # Web app framework

pip install anthropic # Claude AI API client

pip install PyPDF2 # PDF text extraction

pip install python-dotenv # Load .env file
```

Your requirements.txt file:

```
streamlit==1.56.0

anthropic==0.97.0

PyPDF2==3.0.1

python-dotenv==1.2.2
```

■ requirements.txt is like pom.xml in Maven. It lists all packages your project needs. Anyone can install all of them with one command: `pip install -r requirements.txt`

Your .env File — API Key Storage

Create a file called .env in your project folder:

```
ANTHROPIC_API_KEY=sk-ant-your-actual-key-here
```

■■ NEVER put this file on GitHub. Add .env to your .gitignore file. If your API key is exposed publicly, Anthropic will revoke it within minutes and you will be billed for any usage.

V1 Complete Code — Line By Line

Lines 1-2: Load environment variables

```
from dotenv import load_dotenv

load_dotenv()
```

from dotenv import load_dotenv — imports the load_dotenv function from the python-dotenv package. 'from X import Y' means: go to package X, bring function Y into this file.

load_dotenv() — reads your .env file and loads all key=value pairs into the operating system's environment variables. After this line, os.getenv('ANTHROPIC_API_KEY') will return your actual key.

Lines 3-7: Import libraries

```
import os

import streamlit as st

import anthropic

import PyPDF2

import io
```

import os — gives access to operating system functions. We use it for os.getenv() to read environment variables.

import streamlit as st — imports Streamlit and gives it the short alias 'st'. Now instead of streamlit.title() we write st.title(). This is a Python convention for frequently used libraries.

import anthropic — imports the official Anthropic Python SDK. This gives us the Anthropic class to create a client that talks to Claude.

import PyPDF2 — imports the PDF reading library. Used to open PDF files and extract text page by page.

import io — imports Python's built-in input/output utilities. We need io.BytesIO() to treat bytes in memory like a file object.

■ Java equivalent of imports: like import statements at the top of a Java class. You are telling Python which packages to load.

Line 9: Create Claude client

```
client = anthropic.Anthropic()
```

Creates a connection to Claude API. The `Anthropic()` constructor automatically reads `ANTHROPIC_API_KEY` from environment variables (which `load_dotenv()` loaded). You do not need to pass the key manually.

■ This is created once at the top of the file. Every time Streamlit reruns your file (on every user action), a new client is created. This is acceptable for V1 — we optimize this in V2.

Lines 11-19: `extract_text_from_pdf` function

```
def extract_text_from_pdf(pdf_file):

    pdf_reader = PyPDF2.PdfReader(io.BytesIO(pdf_file.read()))

    text = ''

    for page_num, page in enumerate(pdf_reader.pages):

        page_text = page.extract_text()

        if page_text:

            text += f'\n--- Page {page_num + 1} ---\n'

        text += page_text

    return text
```

`def extract_text_from_pdf(pdf_file):` — defines a function. 'def' means define. 'pdf_file' is the parameter — the uploaded file object that Streamlit gives us.

`pdf_file.read()` — reads the raw bytes from the uploaded file. Returns bytes (binary data).

`io.BytesIO(...)` — wraps those bytes in a stream object. PyPDF2 normally expects a file path on disk, but Streamlit gives us bytes in memory. BytesIO is a bridge — it pretends to be a file but lives in memory.

`PyPDF2.PdfReader(...)` — creates a PDF reader object that can access individual pages.

`enumerate(pdf_reader.pages)` — loops through all pages. `enumerate()` gives both the index number (`page_num`) and the page object. Like using a for loop with an index in Java.

`if page_text:` — only add this page's text if it's not empty. Some PDFs have blank pages or image-only pages that return `None`.

`f'\n--- Page {page_num + 1} ---\n'` — an f-string (formatted string). The `{page_num + 1}` is replaced with the actual number. We add 1 because `page_num` starts at 0 (Python is zero-indexed) but humans count from 1.

Lines 21-40: `ask_claude` function

```
def ask_claude(document_text, question, chat_history):
```

```
system_prompt = f'''You are a helpful document assistant.
```

```
Here is the document content:
```

```
{document_text}
```

```
Answer questions based ONLY on this document.
```

```
If answer not in document say: I could not find this.
```

```
Be concise and accurate.'''
```

```
message = client.messages.create(
```

```
model='claude-sonnet-4-5',
```

```
max_tokens=1024,
```

```
system=system_prompt,
```

```
messages=chat_history,
```

```
)
```

```
return message.content[0].text
```

system_prompt — instructions given to Claude before any user message. This tells Claude its role and what it can/cannot do. We inject the entire PDF text here. This is called 'context stuffing' — the simplest form of RAG (Retrieval Augmented Generation).

model='claude-sonnet-4-5' — specifies which Claude model to use. Sonnet balances speed and intelligence. Cheaper than Opus, smarter than Haiku.

max_tokens=1024 — maximum length of Claude's response. 1024 tokens $\approx$ 750 words. Enough for most document answers.

messages=chat_history — sends the full conversation history. This is how we simulate memory with a stateless API. Every call includes all previous Q&A; pairs.

message.content[0].text — Claude returns a list of content blocks. We take the first one (.content[0]) and get its text property.

Why question added to history BEFORE the API call

```
# We add question to history BEFORE calling Claude

st.session_state['chat_history'].append({

    'role': 'user',

    'content': question

})


# Then call Claude with the history that includes the question

answer = ask_claude(document_text, question, chat_history)
```

Claude's API strictly expects alternating user/assistant messages. If we sent the history WITHOUT the current question, Claude would not know what to answer. The question must be the last message in history when sent to Claude.

■ ■ If the API call fails, we remove the question from history using `chat_history.pop()`. This prevents a broken conversation state where Claude sees two consecutive user messages with no assistant response between them.

V2 — Error Handling, Security, Client API Key

V2 transforms the prototype into something real users can use. Three major additions: proper error handling so the app never shows a red crash screen, client API key input so clients pay their own bills, and file validation so bad uploads are caught gracefully.

New: Error handling	Every API error shows a friendly message instead of crashing
New: API key input	Clients enter their own Anthropic key — you pay nothing
New: File validation	Size limit, scanned PDF detection, corrupted file handling
New: Page config	Browser tab title, favicon, centered layout
New: Custom CSS	Rounded corners, card styling, professional look
New: Session state	Cleaner dictionary-based initialization pattern

New Dependency

No new packages for V2. All improvements use existing libraries.

New: Configuration Constants

```
MAX_FILE_SIZE_MB = 10
```

```
MAX_FILE_SIZE_BYTES = MAX_FILE_SIZE_MB * 1024 * 1024
```

Constants defined once at the top in UPPERCASE. If you want to change the limit from 10MB to 25MB — change ONE line, not hunt through code. This is called DRY: Don't Repeat Yourself.

The math: 1 MB = 1024 KB = 1024 × 1024 bytes. So 10 MB = 10,485,760 bytes.

■ Java equivalent: `static final int MAX_SIZE = 10` at the top of your class.

New: Page Config

```
st.set_page_config(  
  
page_title='SmartDocs — AI Document Assistant',  
  
page_icon='■',  
  
layout='centered',
```



```
)
```

Sets the browser tab title, favicon emoji, and page layout. MUST be the first Streamlit call in your file. Streamlit throws an error if anything else runs before this. Like a constructor in Java — it must run first.

New: Session State — Cleaner Pattern

```
# OLD V1 way – repetitive

if 'document_text' not in st.session_state:

    st.session_state['document_text'] = None

if 'chat_history' not in st.session_state:

    st.session_state['chat_history'] = []

# ... repeated for every key


# NEW V2 way – dictionary loop

defaults = {

    'document_text': None,

    'document_name': None,

    'chat_history': [],

    'api_key': '',

    'client': None,

}

for key, val in defaults.items():

    if key not in st.session_state:

        st.session_state[key] = val
```

defaults.items() — returns all key-value pairs from the dictionary. The for loop sets each key only if it doesn't already exist in session state. Adding a new session variable = adding one line to the dictionary.

New: `get_client()` — Smart Key Resolution

```
def get_client():

    key = st.session_state['api_key'].strip()

    if not key:

        try:

            key = st.secrets['ANTHROPIC_API_KEY']

        except Exception:

            key = os.getenv('ANTHROPIC_API_KEY', '')

            if not key:

                return None

    return anthropic.Anthropic(api_key=key)
```

This function checks THREE places for the API key in priority order:

Priority 1	User-entered key in the UI text input (st.session_state['api_key'])
Priority 2	Streamlit Cloud secrets manager (st.secrets) — for deployment
Priority 3	Local .env file via os.getenv() — for local development

.strip() — removes accidental spaces if user pastes key with trailing space. Small detail, big UX improvement.

return None — instead of crashing, returns nothing. The caller (UI code) then decides what to do with None.

■ This single function makes the app work in three environments: your local machine, Streamlit Cloud with your key, and Streamlit Cloud with the client's own key.

New: File Size Validation

```
file_bytes = uploaded_file.getvalue()

file_size_mb = len(file_bytes) / 1024 / 1024

if len(file_bytes) > MAX_FILE_SIZE_BYTES:

    st.error(

        f'File is too large ({file_size_mb:.1f} MB). '
```

```

f'Please upload a PDF under {MAX_FILE_SIZE_MB} MB.'

)

st.info(

'Try these free tools to compress: '

'[SmallPDF](https://smallpdf.com)')

)

st.stop()

```

getvalue() — reads all bytes from the uploaded file at once. Unlike `.read()` which can only be called once, `getvalue()` can be called multiple times on the same file.

:.1f — format specifier. Formats a float to 1 decimal place. 10.4857... becomes 10.5. Clean number for the user.

st.stop() — stops all code execution immediately. Nothing below this line runs. Like `return` but for the entire script.

New: Specific Error Handling

```

try:

    answer = ask_claude(document_text, chat_history)

except anthropic.AuthenticationError:

    st.error('Invalid API key. Please check and try again.')

    st.session_state['chat_history'].pop()

    st.stop()

except anthropic.RateLimitError:

    st.error('Rate limit reached. Wait a moment and try again.')

    st.session_state['chat_history'].pop()

    st.stop()

```

```

except anthropic.APIStatusError as e:

    st.error(f'API error ({e.status_code}): {e.message}')

    st.session_state['chat_history'].pop()

    st.stop()

except Exception as e:

    st.error(f'Something went wrong: {e}')

    st.session_state['chat_history'].pop()

    st.stop()

```

AuthenticationError	Wrong API key entered. Tell user to check their key.
RateLimitError	Too many requests too fast. Tell user to wait.
APIStatusError	Billing/quota issue. Show the error code.
Exception	Catch-all for anything unexpected. Generic message.

chat_history.pop() — removes the last item from the list. When an error occurs, the user's question was added to history but Claude never answered it. This creates a broken state where Claude sees two consecutive user messages. `pop()` removes the unanswered question to keep history clean.

■ Java equivalent: catching specific exceptions (`SQLException`, `NullPointerException`) before the generic `Exception`. Most specific first, generic last.

V3 — Page Range, pdfplumber, Branding

V3 adds power features and professional polish. Page range selection solves the large document problem. pdfplumber replaces PyPDF2 for better text extraction. MIT licensing and footer branding establish ownership.

New: pdfplumber	Better PDF text extraction than PyPDF2
New: Page range	Users select From/To pages — solves large PDF problem
New: Smart re-extract	PDF only re-read when range actually changes
New: Trust caption	Security statement under API key section
New: Footer branding	Your name, LinkedIn, GitHub on every page
New: MIT License	Legal copyright in your name

New Dependency: pdfplumber

```
pip install pdfplumber
```

Updated requirements.txt:

```
streamlit==1.56.0

anthropic==0.97.0

pdfplumber # replaces PyPDF2

python-dotenv==1.2.2

reportlab # for test PDF creation

pytest # for running tests
```

■ PyPDF2 struggled with some PDF formats — words appeared on separate lines making the text unreadable. pdfplumber handles layout better and produces cleaner text extraction.

New: `extract_text_from_pdf` — Updated Signature

```
def extract_text_from_pdf(

    file_bytes: bytes, # bytes not file object
```

```

start_page: int = 1, # default: start from page 1

end_page: int = None # default: go to last page

) -> str:

```

file_bytes: bytes — V1 passed the file object. V3 passes bytes. Why? Streamlit's file object can only be `.read()` once — after that it's empty. By reading bytes once upfront (`getvalue()`), we can call this function multiple times without re-uploading.

: bytes, : int, -> str — type hints. They tell Python (and other developers) what types the function expects and returns. Python doesn't enforce these — they are documentation.

start_page: int = 1 — default parameter value. If you call `extract_text_from_pdf(bytes)` without specifying `start_page`, it defaults to 1.

New: Page Range Loop

```

for i in range(start_page - 1, end_page):

    text = pdf.pages[i].extract_text()

```

start_page - 1 — Python lists are zero-indexed. Page 1 is index 0. Page 5 is index 4. Always subtract 1 to convert human page numbers to Python list indices.

range(start_page - 1, end_page) — if user selects pages 5 to 10, this becomes `range(4, 10) = [4, 5, 6, 7, 8, 9]` = pages 5, 6, 7, 8, 9, 10.

■ Java equivalent: `list.subList(fromIndex, toIndex)`. You give it the range, it returns only that slice.

New: Page Range UI

```

col1, col2 = st.columns(2)

with col1:

    page_from = st.number_input(

        'From page',

        min_value=1,

        max_value=total_pages,

        value=st.session_state['page_from'],

    )

with col2:

```

```

page_to = st.number_input(

    'To page',

    min_value=1,

    max_value=total_pages,

    value=st.session_state['page_to'],

)

```

st.columns(2) — splits the layout into two equal columns side by side. Returns two column objects. We use 'with col1:' and 'with col2:' to put content in each column.

min_value / max_value — prevents user from entering page 0 or page 500 when document only has 50 pages. Built-in validation.

value=st.session_state['page_from'] — remembers the last selected value across reruns. Without this, inputs would reset to 1 every time the user clicks anything on the page.

New: Smart Re-extraction Logic

```

range_changed = (

    page_from != st.session_state['page_from']

    or page_to != st.session_state['page_to']

)

if st.session_state['document_text'] is None or range_changed:

    # Re-extract text

    if range_changed:

        st.session_state['chat_history'] = [] # clear old answers

```

range_changed — compares current input values against last saved values. True only if user actually changed the range.

New file uploaded	document_text is None → extract text → clear chat
Same file, range changed	range_changed = True → re-extract → clear chat
Same file, same range	Nothing to do → skip re-extraction → chat continues

Why clear chat when range changes? Old answers were about pages 1-50. User changed to pages 51-100. Old answers are now wrong — clearing prevents confusion.

New: Footer Branding

```
st.divider()

st.markdown(
    '''
    <div style='text-align: center; color: #6b7280;
    font-size: 0.8rem; padding: 10px'>

    Built by <a href='https://linkedin.com/in/sumit-sharma-dev'
    target='_blank'>Sumit Sharma</a> .

    <a href='https://github.com/SumitSharmaRepository/smardocs'
    target='_blank'>Git Hub</a> .

    2026 SmartDocs

    </div>

    ''',
    unsafe_allow_html=True
)
```

unsafe_allow_html=True — by default Streamlit strips HTML for security. This flag says 'I wrote this HTML myself, it is safe to render'. Only use this for HTML YOU wrote, never for user-submitted HTML.

target='_blank' — opens the link in a new browser tab instead of navigating away from your app.

Automated Testing — test_smartdocs.py

Professional software is tested. SmartDocs has a complete automated test suite with 22 tests across 4 categories. The suite achieved a score of 100/100 — Grade A: Production Ready.

Core Functionality	100/100
Boundary Conditions	100/100
Security	100/100
AI User Simulation	100/100

FINAL SCORE

100/100 — Grade A: Production Ready

Test Categories Explained

Core (30 pts)	Basic text extraction, page ranges, full document. Does the app DO what it claims?
Boundary (25 pts)	Edge cases: empty PDF, corrupted file, very long text, special characters. What breaks it?
Security (30 pts)	API key never exposed, no disk writes, path traversal safe, injection safe.
AI Simulation (15 pts)	Real Claude API calls. Does Claude answer correctly? Handle out-of-scope questions?

Key Testing Concepts

Creating test PDFs in memory

```
from reportlab.pdfgen import canvas

def create_test_pdf(content: str, pages: int = 1) -> bytes:

    buffer = io.BytesIO()

    c = canvas.Canvas(buffer)

    for page in range(pages):

        c.drawString(100, 750, f'Page {page+1}: {content}')

    c.showPage()
```

```
c.save()
```

```
return buffer.getvalue()
```

Test PDFs are created entirely in memory using reportlab — never saved to disk. This is faster, cleaner, and avoids leaving test files behind. `buffer.getvalue()` returns the PDF as bytes.

Running the test suite

```
python test_smartdocs.py
```

The test runner prints **■ PASS** or **■ FAIL** for each test, calculates weighted scores per category, and prints the final score out of 100. Exit code 0 if score ≥ 75 , exit code 1 if below.

■ Industry standard: Exit code 0 = success, exit code 1 = failure. CI/CD systems use this to know if tests passed or failed.

Deploying SmartDocs to Streamlit Cloud

Streamlit Cloud provides free hosting for Streamlit apps. Your app gets a public URL anyone can access. No server management needed.

Step 1 — Push code to GitHub

```
git init

git add .

git commit -m 'Initial commit'

git branch -M main

git remote add origin https://github.com/username/smartdocs.git

git push -u origin main
```

Step 2 — Connect Streamlit Cloud

```
# Go to: share.streamlit.io

# Sign in with GitHub

# Click: New app

# Select: your smartdocs repository

# Main file path: app.py

# Click: Deploy
```

Step 3 — Add API Key to Streamlit Secrets

```
# In Streamlit Cloud dashboard:

# Your app → Settings → Secrets

# Add:

ANTHROPIC_API_KEY="sk-ant-your-actual-key"
```

The API Key Resolution Code — Both Environments

```
try:

    key = st.secrets['ANTHROPIC_API_KEY'] # Streamlit Cloud

except:

    key = os.getenv('ANTHROPIC_API_KEY') # Local .env
```

Local machine	load_dotenv() reads .env → os.getenv() finds the key
Streamlit Cloud	st.secrets reads from their encrypted secrets manager
Both	Same code, works automatically in both places

Your Complete smartdocs/ Folder

```
smartdocs/

■■■ app.py Main application – all UI and logic

■■■ test_smartdocs.py Automated test suite – 100/100 score

■■■ requirements.txt Python dependencies list

■■■ .env API keys – LOCAL ONLY, never on GitHub

■■■ .gitignore Tells Git which files to ignore

■■■ README.md Project description with badges

■■■ CHANGELOG.md Version history V1, V2, V3

■■■ CLAUDE.md Instructions for Claude Code AI assistant

■■■ SDD.md Software Design Document

■■■ LICENSE MIT License – Copyright Sumit Sharma 2026
```

app.py	Your entire application. All code lives here for now.
test_smartdocs.py	22 automated tests. Run with: python test_smartdocs.py
requirements.txt	All Python packages needed. Install with: pip install -r requirements.txt
.env	Your API keys. NEVER commit this to GitHub. In .gitignore.
.gitignore	List of files Git should never track. .env and venv/ are critical.
README.md	What visitors see when they open your GitHub repo.
CHANGELOG.md	Documents what changed in each version and why.
CLAUDE.md	Rules for Claude Code AI assistant when working on this project.
SDD.md	Architecture decisions, design rationale, future roadmap.
LICENSE	MIT License. Anyone can use your code but must credit you.

Quick Reference — Everything You Learned

Python Concepts

<code>import X</code>	Load an installed library into your file
<code>import X as y</code>	Load library X and give it short alias y
<code>def func(param):</code>	Define a reusable function
<code>f'{variable}'</code>	f-string: embed variables inside strings
<code>: type</code>	Type hint: documents expected parameter type
<code>-> type</code>	Return type hint: documents what function returns
<code>for x in range(a,b)</code>	Loop from a to b-1
<code>list.pop()</code>	Remove and return the last item from a list
<code>dict.items()</code>	Return all key-value pairs from a dictionary
<code>.strip()</code>	Remove whitespace from start and end of string
<code>io.BytesIO(bytes)</code>	Wrap bytes in a stream object (pretend file)

Streamlit Concepts

<code>st.session_state</code>	Dictionary persisting across reruns. Only memory that survives.
<code>st.stop()</code>	Immediately stop all code execution.
<code>st.spinner('...')</code>	Show loading animation while code runs inside the with block.
<code>st.columns(n)</code>	Split layout into n equal columns side by side.
<code>st.chat_message(r)</code>	Display a chat bubble for role 'user' or 'assistant'.
<code>st.chat_input('...')</code>	Input box that clears after user hits Enter.
<code>st.expander('...')</code>	Collapsible section. Hidden by default.
<code>st.set_page_config()</code>	Must be first Streamlit call. Sets tab title and favicon.
<code>unsafe_allow_html</code>	Allow raw HTML in <code>st.markdown()</code> . Use only for your own HTML.
<code>st.rerun()</code>	Force an immediate full page rerun.

API & AI Concepts

Stateless API	Claude remembers NOTHING between calls. Send everything every time.
Context stuffing	Put entire document in the prompt. Simplest form of RAG.

System prompt	Instructions to Claude before conversation starts.
Chat history	List of {role, content} dicts. Sent with every API call.
max_tokens	Maximum length of Claude's response. $1024 \approx 750$ words.
Token	~ 0.75 words. Claude's unit of text measurement.
RAG	Retrieval Augmented Generation. Give AI real data to answer from.

VERSION EVOLUTION SUMMARY

How SmartDocs Grew From V1 to V3

V1 Features	PDF upload · Text extraction · Claude API call · Multi-turn conversation · Session state · Clear conversation button
V2 Added	Client API key input · 3-level key fallback · File size validation (10MB) · Scanned PDF detection · AuthenticationError handling · RateLimitError handling · APIStatusError handling · Generic Exception handler · chat_history.pop() on errors · Page config · Custom CSS · Dictionary session state pattern
V3 Added	pdfplumber replacing PyPDF2 · Page range selection · Smart re-extraction (only when range changes) · Total page count display · From > To validation · Large file compression links · File caption with range/size · Trust caption (data privacy statement) · Footer branding with LinkedIn/GitHub · MIT License · Automated test suite (100/100)

The Development Philosophy

V1 → Ship something working

V2 → Make it production safe

V3 → Make it user friendly

V4 → Make it scalable (planned)

V5 → Make it a business (planned)

■ Build the minimum that works. Learn from real users. Add complexity only when needed. This is how all successful products are built — not by imagining every feature upfront.

Final Checklist — What Your Repo Has

■ app.py	Production-ready code with error handling
■ test_smartdocs.py	100/100 automated QA suite
■ requirements.txt	Clean dependency list
■ README.md	Badges, your name, live demo link
■ CHANGELOG.md	V1, V2, V3 documented
■ CLAUDE.md	AI assistant coding instructions
■ SDD.md	Architecture and design rationale

■ LICENSE	MIT — Copyright Sumit Sharma 2026
■ .gitignore	API keys protected
■ Live URL	asksmartdocs-ai.streamlit.app

Built By

Developer	Sumit Sharma
LinkedIn	linkedin.com/in/sumit-sharma-dev
GitHub	github.com/SumitSharmaRepository/smartdocs
Live App	asksmartdocs-ai.streamlit.app
Built	April–May 2026
Score	100/100 — Grade A: Production Ready